

MOODLIFT

Suggest an Activity to Feel Relaxed

Document Type	Software Requirements Specification (SRS)
Project Type	College / University Software Engineering Project
Primary Scope	Frontend-based mood-aware activity recommendation platform
Theme	Green — calm, positive, and wellness-oriented
Version	1.0
Date	16 August 2026

Prepared for academic evaluation, requirement analysis, UI/UX design, implementation planning, testing, and future backend/API expansion.

Table of Contents

Section	Topic
1	Introduction
2	Overall Description
3	User Roles and Stakeholders
4	Functional Requirements
5	Non-Functional Requirements
6	User Interface and UX Requirements
7	Data and Future API Requirements
8	System Architecture and Design
9	Use Cases and User Flows
10	Business Rules
11	Validation and Error Handling
12	Testing Requirements
13	Security, Privacy, and Accessibility
14	Project Constraints and Assumptions
15	Future Enhancements
16	Acceptance Criteria
17	Glossary

1. Introduction

1.1 Purpose. This SRS defines the functional, non-functional, interface, data, architectural, testing, and acceptance requirements for MoodLift. It provides a shared baseline for students, faculty evaluators, UI/UX designers, developers, and testers.

1.2 Product Vision. MoodLift is a simple, modern web application that asks a user how they currently feel and suggests practical activities that may help them relax, refresh, or improve their mood. The first release is intentionally frontend-focused, while the design keeps a clear path for a future database and recommendation API.

1.3 Scope. The MVP covers mood selection, activity recommendation, activity details, filtering, refresh/regeneration of suggestions, responsive UI, and basic user interaction. It does not attempt to diagnose mental-health conditions or provide clinical treatment.

1.4 Intended Audience. College project team, faculty reviewers, software engineering instructors, UI/UX reviewers, developers, and testers.

1.5 Document Conventions. “Shall” indicates a mandatory requirement; “should” indicates a recommended requirement; “may” indicates an optional enhancement.

2. Overall Description

2.1 Product Perspective

MoodLift is a standalone browser-based frontend application. In the MVP, activity data and recommendation logic can be stored locally in JavaScript/JSON. The architecture should isolate data access and recommendation logic so that a REST API and database can be introduced later without redesigning the user interface.

2.2 Core Product Flow

- Open MoodLift → view a welcoming landing screen.
- Select one mood or emotional state.
- System maps the selected mood to suitable activity categories.
- System displays a small set of activity cards.
- User opens an activity to see instructions, duration, difficulty, and expected benefit.
- User may refresh recommendations, change mood, or apply simple filters.

2.3 MVP Feature Set

Feature	Description	Priority
Mood selection	Choose from predefined mood states such as stressed, sad, tired, bored, anxious, or calm.	Must
Recommendation engine	Rule-based mapping between mood and activity categories.	Must
Activity cards	Show activity title, category, duration, difficulty, and short benefit.	Must
Activity details	Show steps, duration, required items, and helpful notes.	Must
Regenerate suggestions	Provide alternative activities for the same mood.	Should
Filters	Filter by duration, category, or difficulty.	Should
Responsive design	Support mobile, tablet, and desktop layouts.	Must
Persistence	Optional local storage for selected mood or favorites.	Could

2.4 Out of Scope for MVP

- Medical diagnosis, therapy, crisis intervention, or clinical recommendations.
- User accounts, social networking, messaging, and community moderation.
- AI/ML-based mood detection from camera, voice, or facial expressions.
- Wearable/device integration and biometric data.
- Paid subscriptions, advertising, and e-commerce.
- Complex analytics dashboards or administrator portals.

3. User Roles and Stakeholders

Role	Responsibilities / Interest
End User	Select mood, explore recommendations, view activity details, and interact with the interface.
Student Developer	Implement UI, recommendation rules, data model, validation, and testing.
UI/UX Designer	Design accessible, calm, responsive, and consistent screens.
Faculty Evaluator	Evaluate completeness, usability, technical quality, testing, and documentation.
Future Admin	Potential future role for maintaining activity content and categories.

3.1 Stakeholder Goals

- Users should receive useful suggestions with minimal effort.
- Developers should be able to add or edit activities without rewriting UI components.
- Faculty should be able to verify clear traceability from requirements to implementation and tests.
- The product should be visually polished without requiring an unnecessarily large backend.

4. Functional Requirements

4.1 Landing and Navigation

FR-001 — The system shall provide a landing page explaining MoodLift in one or two concise sentences.

FR-002 — The system shall provide a clear call-to-action such as “How are you feeling?” or “Find an activity”.

FR-003 — The system shall provide navigation to the home/recommendation experience without requiring login.

4.2 Mood Selection

FR-004 — The system shall display a predefined list of moods/emotional states.

FR-005 — Each mood shall have a clear label and visually distinguishable icon or illustration.

FR-006 — The user shall be able to select one mood at a time in the MVP.

FR-007 — The system shall validate that a mood is selected before generating recommendations.

FR-008 — The system should allow the user to change the selected mood without restarting the application.

4.3 Recommendation Engine

FR-009 — The system shall map each supported mood to one or more activity categories.

FR-010 — The system shall return a small recommendation set, preferably 3–6 activities, to avoid overwhelming the user.

FR-011 — Recommendations shall be selected only from available activity records.

FR-012 — The system shall avoid showing the same activity repeatedly when the user requests regeneration, where enough alternatives exist.

FR-013 — The recommendation logic shall be separated from UI rendering so it can later be replaced by an API or ML service.

4.4 Activity Details

FR-014 — Each activity shall have a unique identifier.

FR-015 — The activity card shall show title, category, approximate duration, and short benefit.

FR-016 — The details view shall show step-by-step instructions.

FR-017 — The details view shall show required items/equipment when applicable.

FR-018 — The user shall be able to return to recommendations without losing the current mood.

4.5 Filters and Interaction

FR-019 — The user should be able to filter recommendations by duration.

FR-020 — The user should be able to filter by activity category and/or difficulty.

FR-021 — The system shall display a friendly empty state when no activity matches the selected filters.

FR-022 — The system should provide a regenerate/refresh action for alternative suggestions.

4.6 Optional Local Persistence

FR-023 — If favorites are implemented, the system may store favorite activity IDs in browser local storage.

FR-024 — The system shall continue to work if local storage is unavailable or disabled.

5. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Usability	A first-time user should understand the primary action without instructions.
NFR-002	Performance	For local MVP data, the main recommendation interaction should feel immediate, with no unnecessary blocking delay.
NFR-003	Responsive	Layout shall adapt to mobile, tablet, and desktop widths.
NFR-004	Accessibility	Interactive controls shall be keyboard reachable and have visible focus states.
NFR-005	Accessibility	Text and controls should maintain readable contrast and meaningful labels.
NFR-006	Maintainability	Components, data, and recommendation rules should be separated into logical modules.
NFR-007	Compatibility	The MVP should work on current versions of Chrome, Edge, and Firefox.
NFR-008	Reliability	Invalid or missing activity data shall not cause the entire application to crash.
NFR-009	Scalability	Activity data should be structured so a future API/database can replace local data with minimal UI changes.

6. User Interface and UX Requirements

6.1 Visual Design

- Use green as the primary brand color, supported by soft neutral backgrounds and high-contrast text.

- Use rounded cards, generous spacing, simple illustrations/icons, and restrained animation to create a calm experience.
- Use a consistent button hierarchy: primary action, secondary action, and text/icon actions.
- Avoid excessive visual density; recommendations should be scannable.

6.2 Suggested Screens

Screen	Purpose	Key UI Elements
Home	Introduce MoodLift and start the journey.	Logo, headline, CTA, calming visual, brief explanation.
Mood Selection	Capture current mood.	Mood cards/chips, selected state, Continue button.
Recommendations	Present suggested activities.	Mood summary, activity cards, filters, regenerate action.
Activity Details	Explain one activity.	Title, benefit, duration, difficulty, steps, back button.
Empty/Error State	Handle no matches or data problems.	Friendly message, retry/change mood action.

6.3 Responsive Breakpoints

- Mobile: single-column cards and touch-friendly controls.
- Tablet: two-column recommendation grid where space permits.
- Desktop: centered content container with two-to-four activity cards per row depending on card width.

7. Data and Future API Requirements

7.1 Activity Data Model

Field	Type	Required	Purpose
id	String/UUID	Yes	Unique activity identifier.
title	String	Yes	Activity name.
description	String	Yes	Short explanation.
category	Enum/String	Yes	Breathing, music, walk, creative, mindfulness, etc.
durationMinutes	Integer	Yes	Approximate time required.
difficulty	Enum	Yes	Easy, Medium, or similar.
benefit	String	Yes	Expected non-clinical benefit.
steps	Array	Yes	Ordered instructions.
items	Array	No	Optional materials.
moods	Array	Yes	Moods for which the activity is suitable.
imageUrl	String	No	Optional activity visual.

7.2 Suggested Mood Set

A practical MVP can begin with: **Stressed, Sad, Tired, Bored, Anxious, and Calm**. The exact labels should be finalized during UI/UX design and user testing.

7.3 Future API Contract

Endpoint	Method	Purpose
/api/moods	GET	Return supported moods.
/api/activities	GET	Return activities with optional filters.
/api/recommendations	POST	Accept mood/filter context and return ranked recommendations.

The frontend should access data through a small service/data-access layer rather than hard-coding activity objects directly inside visual components.

8. System Architecture and Design

8.1 Recommended MVP Architecture

Layer	Responsibility	MVP Technology Option
Presentation	Pages, components, responsive styling, interaction.	HTML/CSS/JavaScript or React
Application Logic	Mood state, filtering, recommendation orchestration.	JavaScript modules / React hooks
Recommendation	Rule-based mood → activity matching.	Local rule module
Data Access	Load activity records and abstract data source.	JSON/local data; future REST API
Persistence	Optional favorites/preferences.	localStorage; future database

8.2 Component Structure (Suggested)

- App / Router
- Header / Navigation
- MoodSelector
- MoodCard
- RecommendationList
- ActivityCard
- ActivityDetails
- FilterBar
- EmptyState / ErrorState
- recommendationService
- activityService

8.3 Recommendation Logic

A simple rule-based approach is recommended for the college MVP. Each activity contains a list of suitable moods. When a user selects a mood, the engine filters activities whose mood list contains the selected value, optionally applies duration/category/difficulty filters, removes recently shown IDs, and returns the top 3–6 results. This provides explainable behavior and is easy to test.

9. Use Cases and User Flows

ID	Use Case	Actor	Precondition	Outcome
UC-01	Start MoodLift	User	Application is accessible.	Home screen displayed.

ID	Use Case	Actor	Precondition	Outcome
UC-02	Select Mood	User	Mood selection screen open.	Mood stored in current session.
UC-03	Get Recommendations	User	Mood selected.	Relevant activity cards displayed.
UC-04	View Activity	User	Activity exists.	Detailed instructions displayed.
UC-05	Regenerate	User	Recommendations exist.	Alternative suggestions displayed when available.
UC-06	Filter Activities	User	Recommendations exist.	Matching activities displayed.

9.1 Primary User Flow

Home → Select Mood → Validate Mood → Generate Recommendations → Browse Cards → View Details → Return / Regenerate / Change Mood

9.2 Alternative Flow: No Matching Activity

Apply filters → No results → Show empty state → User clears filters, changes mood, or returns to all recommendations.

10. Business Rules

BR-01 — A recommendation must belong to the selected mood mapping.

BR-02 — The system shall not display activities missing mandatory user-facing content.

BR-03 — Duration filters shall use non-negative integer minute values.

BR-04 — If fewer than the target number of recommendations are available, the system shall display all valid matches rather than fabricate results.

BR-05 — Regeneration shall prefer activities not shown in the current recommendation set.

BR-06 — MoodLift content shall be framed as general wellness/relaxation suggestions, not medical advice.

BR-07 — The user shall not need an account to access core MVP functionality.

11. Validation and Error Handling

- No mood selected: disable/guard the recommendation action and explain what the user needs to choose.
- Invalid activity record: skip the record and log a development-friendly error rather than breaking the UI.
- No recommendations: show a friendly empty state with clear next actions.
- Data loading failure in a future API version: show retry and fallback messaging where possible.
- Invalid filter combination: reset or explain the conflicting filter instead of returning a blank unexplained screen.

Errors should be understandable to users and detailed enough for developers during development. Raw stack traces should never be shown in the production UI.

12. Testing Requirements

Test Area	Examples	Expected Result
Functional	Select each supported mood.	Correct selected state and recommendations.

Test Area	Examples	Expected Result
Recommendation	Verify mood-to-activity mappings.	Only valid matching activities returned.
Filtering	Apply duration/category/difficulty filters.	Only matching records remain.
Navigation	Open details and return.	Correct activity and preserved session mood.
Edge Cases	No matching activity, missing image, empty steps.	Graceful fallback without crash.
Responsive	Test mobile/tablet/desktop widths.	No clipping or unusable controls.
Accessibility	Keyboard navigation and focus states.	All core actions accessible.
Compatibility	Chrome, Edge, Firefox.	Core MVP behavior works consistently.

12.1 Minimum Test Levels

- Unit testing: recommendation rules, filters, data validation utilities.
- Integration testing: mood selection → recommendation rendering.
- UI testing: navigation, buttons, cards, empty states, responsive behavior.
- System testing: complete user flow from home to activity details.
- User acceptance testing: representative students evaluate usefulness, clarity, and ease of use.

13. Security, Privacy, and Accessibility

13.1 Security

- The MVP should avoid collecting personally identifiable information unless a future requirement explicitly needs it.
- If an API is introduced, validate all incoming parameters server-side.
- Do not trust client-side filters or IDs when backend persistence is added.
- Do not expose API keys or secrets in frontend source code.

13.2 Privacy

- Core functionality should work without account creation.
- Optional local preferences should remain on the user's device unless the user explicitly opts into future cloud features.
- Any future analytics must define what is collected, why it is collected, and how long it is retained.

13.3 Accessibility

- Use semantic HTML and meaningful button/link labels.
- Provide visible keyboard focus indicators.
- Do not rely only on color to communicate mood selection or status.
- Provide text alternatives for meaningful images/icons.
- Maintain readable font sizes, spacing, and contrast.

14. Project Constraints and Assumptions

Item	Constraint / Assumption
Team	Small student development team with limited project time.
Platform	Web browser; no native mobile app required for MVP.
Backend	Not required for the first release; architecture must allow future integration.
Data	Initial activity dataset is curated manually and stored locally.
Content	Suggestions are general wellness activities, not clinical treatment.
Connectivity	Core MVP may work with locally bundled data after the app is loaded.
Technology	Implementation may use HTML/CSS/JavaScript or React depending on team skill and course requirements.

15. Future Enhancements

- User accounts and synchronized preferences.
- Favorites and activity history.
- Backend REST API with database-backed activity management.
- Admin panel for adding/editing activities.
- Personalized recommendation ranking using user feedback.
- Optional AI/ML recommendation layer after sufficient usage data is available.
- Mood journal and trend visualization, with strong privacy controls.
- Multilingual support, including Hindi and English.
- Progressive Web App (PWA) support.
- Accessibility audit and WCAG-aligned improvements.

16. Acceptance Criteria

ID	Acceptance Criterion
AC-01	A user can open the application and understand how to start without external instructions.
AC-02	A user can select a supported mood and receive relevant activities.
AC-03	Every displayed activity contains sufficient information to understand and perform it.
AC-04	Filters, when implemented, update the recommendation list correctly.
AC-05	No-result and error scenarios are handled with usable messages.
AC-06	The UI is usable on mobile and desktop screen sizes.
AC-07	Core interactions are keyboard accessible and visually understandable.
AC-08	Recommendation logic is separated sufficiently to allow future API replacement.
AC-09	The implementation passes the documented functional and system test cases.

17. Glossary

Term	Meaning
Mood	A user-selected emotional state used as recommendation input.
Activity	A practical relaxation, refreshment, or wellness-oriented action suggested by MoodLift.
Recommendation	An activity selected because it matches the user's current mood and optional filters.
MVP	Minimum Viable Product: the smallest useful release satisfying the core project objective.
Rule-based	A deterministic recommendation approach using predefined mappings and conditions.
REST API	A web service interface that can later provide moods, activities, and recommendations.
Responsive UI	Interface that adapts to different screen sizes and input contexts.

End of Document

MoodLift SRS v1.0 is intentionally scoped for a realistic college software project while providing a professional foundation for implementation, testing, faculty evaluation, and future backend expansion.